

Smart Blind Assistant

Rapport d'avancement — Modifications, Ameliorations & Propositions

Etat actuel du projet

Module	Fichier	Statut	Notes
Model Loader	cv/core/model_loader.py	DONE	Singleton - charge YOLO une fois
Detecteur	cv/detector.py	DONE	Obstacles + position + distance
Finder	cv/finder.py	DONE	Recherche objet precis
Localizer	cv/localizer.py	DONE	Detection de piece
API	main.py	DONE	3 routes CV branchees
Fine-tuning	cv/train.py	A FAIRE	Apres collecte dataset
NLP	nlp/intent.py	A FAIRE	Apres fine-tuning
Navigation	navigation/	A FAIRE	Phase finale

Phase 1 — Setup et test initial

1.1 Test YOLO de base

Avant de coder quoi que ce soit, un test rapide avec yolov8n.pt a ete effectue sur des images reelles de l'environnement. Resultats sur la premiere image :

Objet detecte	Confiance	Observation
tv	0.50	Utile pour localizer (salon)
potted plant	0.33	Confiance faible
vase (x3)	0.29 - 0.60	Confiances basses

Conclusion : YOLO de base detecte les objets du quotidien mais avec des confiances basses. Un fine-tuning sera necessaire pour un usage fiable en conditions reelles.

1.2 Analyse des classes COCO vs besoins du projet

Classes manquantes identifiees apres analyse :

Objet manquant	Raison	Priorite
robinet	Absent de COCO	Haute
prise de courant	Absent de COCO	Haute
flamme	Absent de COCO	Haute
escalier	Dans COCO mais peu fiable	Haute
douche	Absent de COCO	Moyenne
armoire / closet	Approximatif dans COCO	Moyenne

serviette	Absent de COCO	Basse
table de nuit	Absent de COCO	Basse

Phase 2 — Architecture et decisions de conception

2.1 Separation des responsabilites

Decision cle : les 3 modules CV (detector, finder, localizer) partagent le meme modele YOLO via un singleton. Chaque module a une responsabilite unique :

Module	Role	Post-processing
detector.py	Detecte tous les obstacles	Filtre par proximite + position
finder.py	Cherche un objet precis	Filtre par classe cible demandee
localizer.py	Reconnait la piece	Score par signatures d'objets
model_loader.py	Charge YOLO une seule fois	Singleton - partage entre modules

2.2 Choix : fonction vs classe pour finder.py

Proposition initiale : finder.py comme classe (ObjectFinder). Modification apportee : passage en fonction simple find_object(image, target) car le module n'a pas besoin d'etat interne. Plus simple, plus lisible.

Bonne pratique : utiliser une classe seulement quand on a besoin de garder un etat. Ici, get_model() gere deja le singleton — une fonction suffit.

Phase 3 — Ameliorations de detector.py

3.1 Correction des coordonnees (float -> int)

Probleme identifie : YOLO retourne des coordonnees en float (ex: x1=0.30 au lieu de 0). OpenCV necessite des entiers pour dessiner les boites. Correction appliquee :

```
# Avant (incorrect)
x1, y1, x2, y2 = box.xyxy[0].tolist()

# Apres (correct)
x1, y1, x2, y2 = [int(v) for v in box.xyxy[0].tolist()]
```

3.2 Probleme identifie : estimation de distance naive

Probleme : le ratio brut bbox/image desavantage les petits objets. Un vase proche semblait 'loin' compare a un lit lointain. Exemple concret :

Objet	Taille reelle	Ratio brut	Distance affichee (avant)	Distance correcte
lit (loin)	Grand	15%	proche	loin
vase (proche)	Petit	4%	loin	tres proche

3.3 Solution retenue : Taille de reference par classe (Option 1)

Principe : normaliser le ratio de chaque detection par une taille de reference propre a sa classe. Un vase 'normal' proche = 4% de l'image, un lit 'normal' proche = 25%. On compare le ratio observe a cette reference :

```
ratio = bbox_area / img_area
ref = TAILLE_REF.get(cls_name, 0.08) # 0.08 par default si classe inconnue
ratio_normalise = ratio / ref

ratio_normalise > 2.0 → 'tres proche'
ratio_normalise > 0.8 → 'proche'
sinon → 'loin'
```

Robustesse : si l'objet n'est pas dans TAILLE_REF, .get() retourne 0.08 par default. Pas d'erreur, juste une estimation moins precise.

3.4 Comparaison des approches de distance

Approche	Precision	Vitesse	Classes	Complexite	Decision
Ratio brut	Faible	Tres rapide	Toutes	Simple	Abandonne
Taille ref (Option 1)	Moyenne	Tres rapide	Toutes	Simple	Retenu
MediaPipe Objectron	Moyenne	Moyen	~9 seulement	Moyen	Ecarte
MiDaS Depth	Bonne	Lent	Toutes	Moyen	Phase 4

MediaPipe ecarte car il ne supporte que ~9 classes, insuffisant pour la liste d'objets du projet. MiDaS envisageable en Phase 4 si la precision s'avere insuffisante en conditions reelles.

Phase 4 — Modules CV completes

4.1 detector.py — Version finale

Retourne pour chaque obstacle :

Champ	Type	Description
objet	str	Nom de la classe detectee
confiance	float	Score de confiance YOLO (seuil ≥ 0.4)
position	str	gauche / centre / droite
distance	str	tres proche / proche / loin
x1, y1, x2, y2	int	Coordonnees de la bounding box

4.2 finder.py — Recherche d'objet precis

Implementation en fonction simple (find_object) plutot qu'en classe. Si plusieurs instances du meme objet sont detectees, retourne celle avec la meilleure confiance. Retourne None si l'objet n'est pas trouve.

Comportement dans le test : meme si l'objet n'est pas trouve, l'image s'affiche avec le texte 'Objet non trouve' en rouge. L'utilisateur a toujours un retour visuel.

4.3 localize.py — Reconnaissance de piece

Approche par regles et scores. Chaque piece est definie par une signature d'objets. Certains objets ont un poids superieur (objets dominants) :

Piece	Objets signature	Objet dominant (poids)
Salon	couch, tv, remote, potted plant	couch (x2), tv (x2)
Cuisine	refrigerator, oven, sink, bottle, bowl	refrigerator (x3)
Chambre	bed, clock, laptop	bed (x3)
Salle de bain	toilet, sink, toothbrush	toilet (x3)
Bureau	laptop, keyboard, mouse, monitor	laptop (x2)
Couloir	door, suitcase, backpack	aucun dominant

Phase 5 — Integration API FastAPI

Routes CV branchees dans main.py

Route	Methode	Input	Module appele	Statut
/detect-obstacle	POST	image	detector.detect()	Branche
/find-object	POST	image + target	find_object()	Branche
/where-am-i	POST	image	localize()	Branche
/navigate	POST	-	navigation/ (a venir)	Mock
/sos	POST	-	safety/sos.py (a venir)	Mock

Mecanisme de sauvegarde temporaire : chaque image recue est sauvegardee dans temp/image.jpg avant d'etre traitee. Suffisant pour traitement image par image.

Compatibilite temps reel

Les modules ont ete concus pour etre compatibles temps reel avec des changements minimes. YOLO accepte nativement un frame numpy (camera) ou un path (fichier). Changements requis pour passer en temps reel :

Fichier	Changement requis	Effort
cv/detector.py	Changer signature : image_path: str → image	Minimal
cv/finder.py	Changer signature : image_path: str → image	Minimal
cv/localizer.py	Changer signature : image_path: str → image	Minimal
main.py ou test/	Ajouter boucle cv2.VideoCapture(0)	Faible

Prochaines etapes

Etape	Description	Priorite
Fine-tuning (Phase 2)	Collecter dataset via Roboflow Universe pour les ~8 classes manquantes. Entraîner sur yolov8n.pt → best.pt	HAUTE
Integration NLP (Phase 3)	Coder nlp/intent.py et intents.json Connexion CV + NLP dans main.py "trouve mon telephone" → find_object()	HAUTE
Temps reel (Phase 3)	Adapter les 3 modules pour accepter frames Ajouter boucle VideoCapture(0)	MOYENNE
Navigation (Phase 4)	Brancher navigation/ dans main.py Graphe de pieces + instructions vocales	MOYENNE
MiDaS Depth (Phase 4)	Integration optionnelle si Option 1 s'avere insuffisante en conditions reelles	BASSE

Prochaine action immediate : collecter le dataset sur Roboflow Universe pour les classes manquantes (robinet, prise, flamme, escalier, douche, armoire). 100-200 images par classe suffisent pour un fine-tuning léger.